Assignment 3

Due Date: 25 October 2026

- This assignment may be completed **individually or in a group of exactly two students**.
- No late submissions will be accepted. Submissions that do not comply with the specifications in this document will not be marked and will receive a zero grade.
- You may use generative AI tools to help with this assignment, but you should generate the minimal amount of code needed to implement the required functionality. You must be able to **explain your code** and reproduce the prompts you used if asked to during the viva. **This is not a one-shot assignment: it requires several components to be built and tested separately before they come together into a working system. Use the tools, but make sure you understand what they produced.**
- Submit a single **private** GitHub repository link on Google Classroom containing all code, documentation, and test cases, **or** a .zip file with the same contents.
- A short Loom video demoing your system is optional but recommended. Your README.md must contain every item listed in the Deliverables section of this document. A group that builds a working system but ships an incomplete README will lose marks on completeness.
- This assignment builds directly on your own group's codebase from the previous assignment in this series. You must continue using the **same business use case** you selected earlier.

Introduction

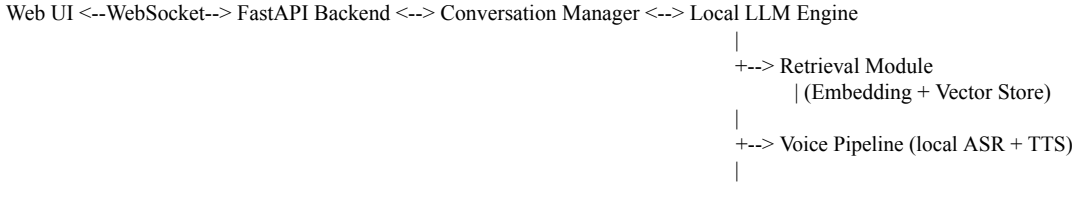
Your assistant can already hold a conversation and ground its answers in your domain's documents. In this final assignment you will give it a voice and the ability to act: local speech-to-text and text-to-speech in a real-time pipeline, plus three tools the LLM can call. Everything from the earlier assignments (streaming, memory, RAG, concurrent users) must work the same way in voice mode.

Objective

- Add local speech-to-text so users can speak to the assistant, with audio streamed to the backend.
- Add local text-to-speech that starts speaking before the full LLM response is ready.
- Implement three tools of your choice that the LLM can call, with timeouts and graceful error handling.
- Make streaming, memory, RAG, and tools work in voice mode exactly as they do in text mode.
- Keep latency close to real time with at least two concurrent users, running entirely locally.

System Architecture Overview

You are extending the Assignment 2 architecture with a local voice pipeline (ASR and TTS) and a tool orchestrator alongside the retrieval module. The existing text WebSocket API contract must still not change; audio is added alongside it.



+--> Tool Orchestrator
+--> Tool 1
+--> Tool 2
+--> Tool 3

Tasks

Phase I — Speech-to-Text

Let the user speak instead of typing, using a local ASR model (no cloud or browser speech APIs). It must:

- Stream microphone audio to the backend over the WebSocket in small chunks.
- Show the transcript, detect the end of speech, and produce a final transcript.
- Pass the final transcript to the same conversation manager used for typed text.

Phase II — Text-to-Speech

Speak the assistant's replies using a local TTS model (no cloud services). It must:

- Start synthesizing sentence by sentence as LLM tokens stream, without waiting for the full response.
- Stream audio back to the browser in chunks and play it in order, while the text still appears in the chat.

Phase III — Voice with Streaming, Memory, and RAG

Voice is not a separate system. Spoken turns must use the same LLM streaming, shared conversation history, and RAG pipeline as typed turns, and text-only chat must keep working.

Phase IV — Tools

Implement three tools useful for your domain. Some starting suggestions of tools are listed below, pick rest of them yourself (a weather tool, a general calculator, and a scheduling/calendar tool are some examples of tools). You are free to substitute your own tools entirely, as long as they are genuinely useful for the business:

- A weather tool that fetches current weather for a given city using a free API (e.g., OpenWeatherMap).
- A calendar tool that reads and writes events to a local calendar file or Google Calendar API.
- A calculator tool that evaluates mathematical expressions.
- A stock price tool that retrieves live or delayed stock prices.
- A news summarisation tool that pulls headlines from RSS feeds.
- A database query tool that searches a product catalogue or a local SQLite database.
- Each tool needs a clear name, a description, and a JSON schema for its arguments.
- The orchestrator must detect when a tool is needed, call it, and pass the result back to the LLM for the final answer (prompt-based function calling or a library; be ready to explain it in the viva).
- Tool calls must be asynchronous, with a timeout, and failures must produce a sensible message, not a hang or crash.
- Tools must work from spoken requests, and their results must be spoken back.

Phase V — Real-Time Behaviour

Measure and report per voice turn: ASR latency, retrieval/tool latency, LLM first-token latency, and TTS first-audio latency. The target is close to real time: ideally around one second and no more than about two seconds from end of speech to first audio (retrieval and tool calls together should add no more than about two seconds before generation starts). The system must handle at least two concurrent users, text or voice, without one blocking another.

Evaluation Criteria

Correctness & Completeness	**50%**
Viva (code walkthrough, live Q&A, reproduce your prompts, voice pipeline, tool relevance to chosen domain)	**25%**
Real-time Behaviour & Performance (latency, streaming, failure handling)	**25%**
Total	**100%**
Bonus — see the Bonus section below	**up to +10%**

Deliverables & Submission Guidelines

Submit a single Private Github repository (or a .zip) containing:

- The voice pipeline code (browser audio capture and playback, local ASR and TTS modules, streaming orchestration) and setup instructions for the local models.
- The tool orchestrator code and all three tool implementations, each with its schema documented.
- A final **README.md** covering the complete three-assignments: an up-to-date architecture diagram, your model choices (LLM, ASR, TTS), your document collection and retrieval setup, each tool's name, description, schema, and an example of the LLM invoking it, your latency benchmarks (ASR, retrieval, tools, LLM, TTS, and end of speech to first audio), setup instructions, and known limitations.

Bonus (up to +10%)

Optionally add the following for bonus credit on this assignment:

- **CRM tool:** a tool that remembers users across sessions, as described below.

The CRM tool must be exposed as something the LLM can call, with a clear name, description, and argument schema. At minimum it must:

- Store user information (name, contact details, preferences, past interactions) keyed by a session or user ID.
- Retrieve that information at the start of a returning session, so the assistant can greet the user by name and recall what they asked for last time.
- Update the record during the conversation when the user provides new information.

A simple persisted key-value store (a JSON file or SQLite) is enough. The important part is that the LLM calls it correctly, not how fancy the storage layer is. The CRM tool must also work in voice mode.

Honor Policy

Cheating in code, datasets, or reports will result in an immediate zero grade and will be reported to the academic

disciplinary committee. All work must be original to your group.